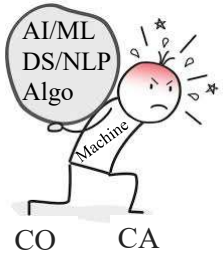
CS-2710-1 Spring 2026

Computer Organization and Systems (COS)



Lecture #12

13 March 2026

Ashoka University

BBB

Integer Multiplication (unsigned)

multiplicand → 1000

multiplier → × 1001

```
  1000
 0000
0000
1000
```

add

product → 1001000

A_i and B_i are all 0 or 1

$$
\begin{array}{rrrrrrrr}
 & & & & A_3 & A_2 & A_1 & A_0 \\
 & & & & \times\ B_3 & B_2 & B_1 & B_0 \\
\hline
 & & & & A_3B_0 & A_2B_0 & A_1B_0 & A_0B_0 \\
 & & & A_3B_1 & A_2B_1 & A_1B_1 & A_0B_1 & \\
 & & A_3B_2 & A_2B_2 & A_1B_2 & A_0B_2 & & \\
+ & A_3B_3 & A_2B_3 & A_1B_3 & A_0B_3 & & & \\
\hline
\end{array}
$$

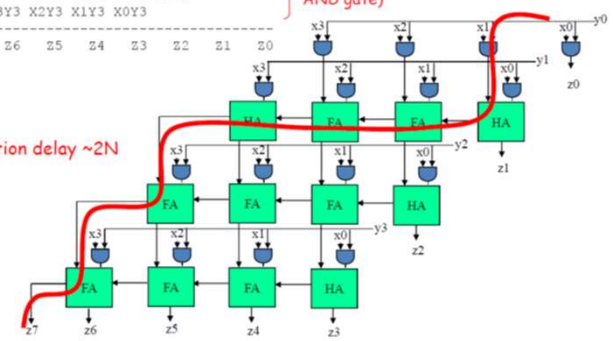
AB_i called a "partial product"

add

Multiplying N-bit number by M-bit number gives (N+M)-bit result

Combinational Multiplier (unsigned)

```
                X3   X2   X1   X0   ← multiplicand
              * Y3   Y2   Y1   Y0   ← multiplier
              -------------------
              X3Y0 X2Y0 X1Y0 X0Y0   Partial products, one for each bit in
+           X3Y1 X2Y1 X1Y1 X0Y1     multiplier (each bit needs just one
+         X3Y2 X2Y2 X1Y2 X0Y2       AND gate)
+       X3Y3 X2Y3 X1Y3 X0Y3
-------------------------------------
  Z7   Z6   Z5   Z4   Z3   Z2   Z1   Z0
```



➢ Propagation delay ~2N

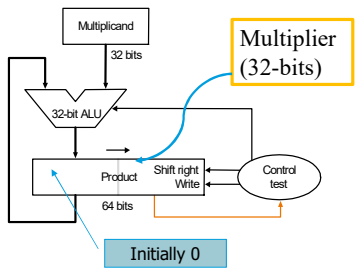
Hardware Multiplier (Shift + Add): Actions

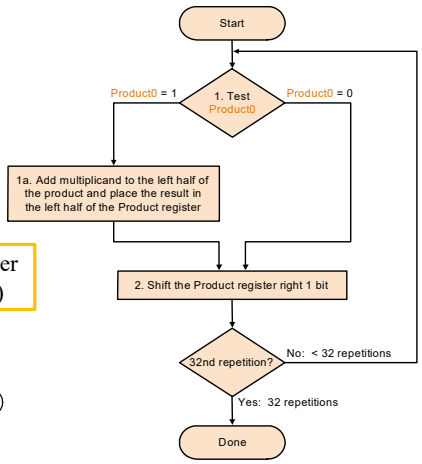
multiplicand → 1000

multiplier → × 1001

```
  1000
 0000
0000
1000
```

product → 1001000





Multiplication Example

0010 × 0110 = ? 0010 (+ 2, multiplicand); 0110 (+ 6, multiplier)

Itera-tion	multi-plicand	Orignal algorithm	
		Step	Product
0	0010	Initial values	0000 0110
1	0010	1:0 ⇒ no operation	0000 0110
	0010	2: Shift right Product logical shift	0000 0011
2	0010	1a:1⇒ prod = Prod + Mcand	0010 0011
	0010	2: Shift right Product	0001 0001
3	0010	1a:1⇒ prod = Prod + Mcand	0011 0001
	0010	2: Shift right Product	0001 1000
4	0010	1:0 ⇒ no operation	0001 1000
	0010	2: Shift right Product +12	0000 1100

Signed Multiplication

- Recall
  - For $p = a \times b$, if either $a < 0$ or $b < 0$, then $p < 0$
  - If ($a < 0$ and $b < 0$) or ($a > 0$ and $b > 0$) then $p > 0$
  - Hence **sign(p) = sign(a) ⊕ sign(b)**
- Hence
  - Convert multiplier and multiplicand to positive number each
  - Multiply two positive numbers
  - Compute sign, convert product accordingly

Booth's Encoding
(valid for signed multiplication as well)

- Recall old trick

  Example: 123454 × 9
  - 123454 × 9 = 123454 × (10 – 1) = 1234540 – 123454
- Booth's algorithm applies the same principle
  - in binary we have just '1' and '0'

Booth's Encoding in Binary Domain

- Multiply x by 01111

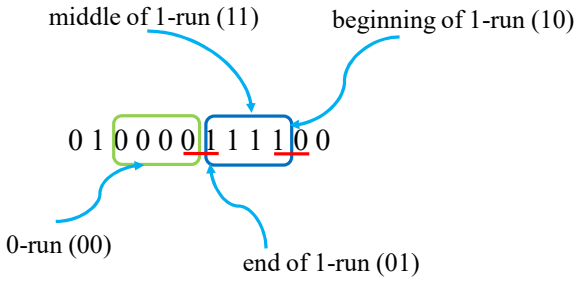
x (multiplicand); 01111 (multiplier)

=> $x \times 15$ (needs four add cycles, 1 shift cycle)

Search for a run of '1' bits in the multiplier
  - e.g. '01111' has a run of 4 consecutive '1' bits
  - Multiplying by '01111' (15 in decimal) is equivalent to multiplying by 16 and subtracting once, since

    $x \times 15 = x \times (16 - 1) = 16x - x$ =>(shift-left x, four times) - x
- Hence, iterate right to left and look for "runs of 1":
- $x \times 1111 = x \times (10000 - 1) = (x \times 2^4) - x$
- needs 1 subtract, 1 add, four shifts!

Booth's Encoding for Multiplier



Booth's Algorithm

Current bit	Bit to right	Explanation	Example (multiplier)	Operation
1	0	Begins run of '1'	00001111000	Subtract/shift
1	1	Middle of run of '1'	00001111000	Nothing/shift

Booth's algorithm: Example

0010 × 1101 = ? 0010 (+ 2, multiplicand); 1101 (- 3, multiplier)

arithmetic shift

op performed in the left half

initialization

Itera-tion	multi-plicand	Booth's algorithm	
		Step	Product
0	0010	Initial values	0000 1101 0
1	0010	1c: 10⇒ prod = Prod - Mcand	1110 1101 0
	0010	2: Shift right Product	1111 0110 1
2	0010	1b: 01⇒ prod = Prod + Mcand	0001 0110 1
	0010	2: Shift right Product	0000 1011 0
3	0010	1c: 10⇒ prod = Prod - Mcand	1110 1011 0
	0010	2: Shift right Product	1111 0101 1
4	0010	1d: 11 ⇒ no operation	1111 0101 1
	0010	2: Shift right Product	1111 1010 1

- 2

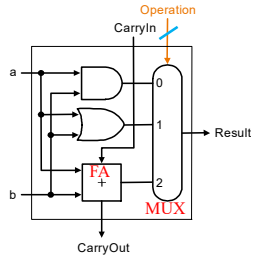
start of 1-run

end of 1-run

-6

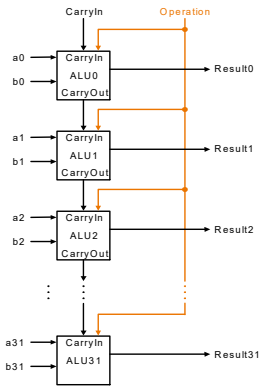
Building a 32-bit ALU capable of executing MIPS instructions

Building a 32-bit ALU to implement *and*, *or*, *add*/*sub*, and *slt* instructions



width of the operation-bus?
= 2

1-bit ALU for performing AND, OR, ADD

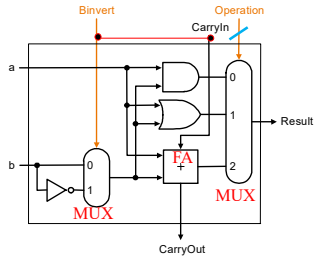


32-bit ALU for performing AND, OR, ADD

What about subtraction (a – b)?

- Two's complement approach: just negate *b* and add to *a*.
- How do we negate?

- Use XOR or MUX to realize 2's complement:

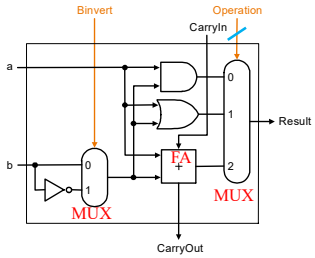


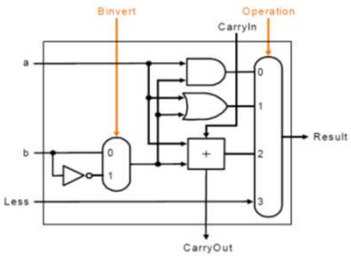
1-bit ALU for performing AND, OR, ADD, SUB

Supporting *slt*

slt $t3, $t1, $t2

$t3 ←1 if $t1 < $t2,
and 0 otherwise





- Can we figure out the idea?

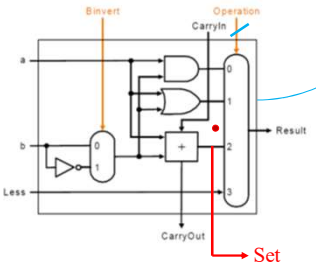
Supporting *slt*

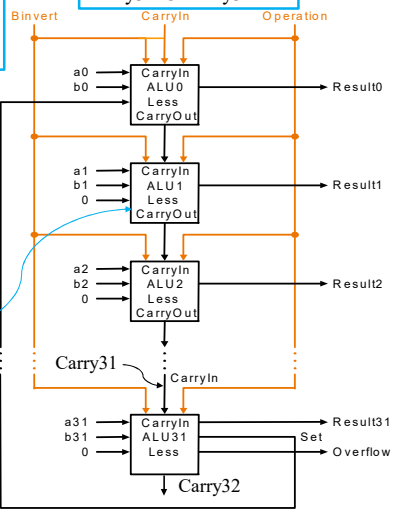
slt $t3, $t1, $t2

$t3 ←1 if $t1 < $t2,
and 0 otherwise;
i.e., when $t1 < $t2,
$t3: 00……001

$t1 → A
$t2 → B
Result → $t3

Overflow =
Carry32 ⊕ Carry31

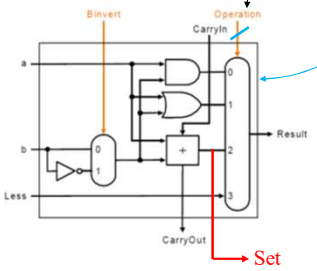


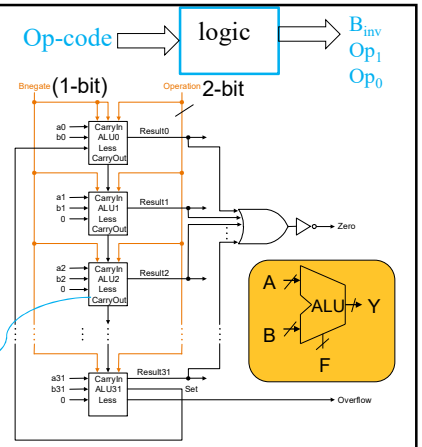


Implementing MIPS instructions

- Control lines
  B_{inv} Op_1 Op_0

```
000 = and
001 = or
010 = add
110 = sub
111 = slt
```





• Note: *Zero* = 1 when the result is zero!

This ALU implements MIPS *add*, *sub*, *and*, *or*, *slt*, and *beq*

Processor (CPU) Design

- We're ready to implement the MIPS "core"
  - load-store instructions: `lw, sw`
  - reg-reg instructions: `add, sub, and, or, slt`
  - control flow instructions: `beq`

MIPS Instruction Execution Cycle

IF → ID/RR → EX → Mem → WB

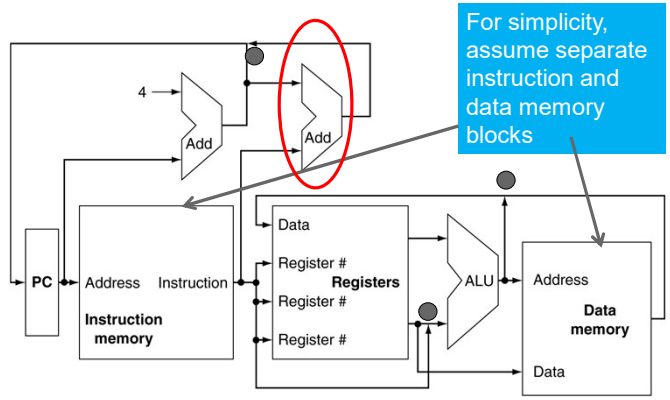
Access memory for load/store;

PC ← target address or PC + 4;

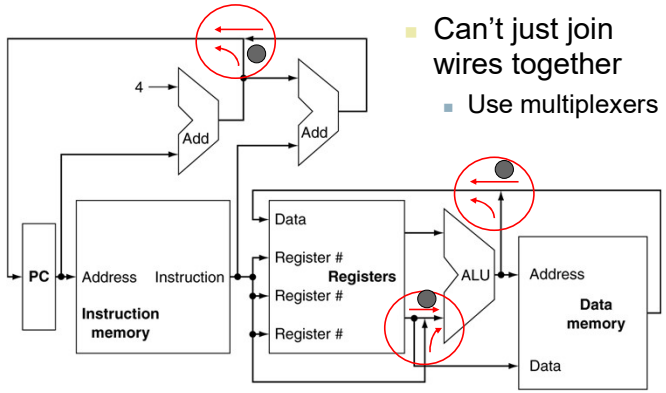
Fetch next instruction from memory address pointed by PC

Two types of functional units:
- elements that operate on data values (combinational)
- elements that contain state (sequential)

For simplicity, assume separate instruction and data memory blocks



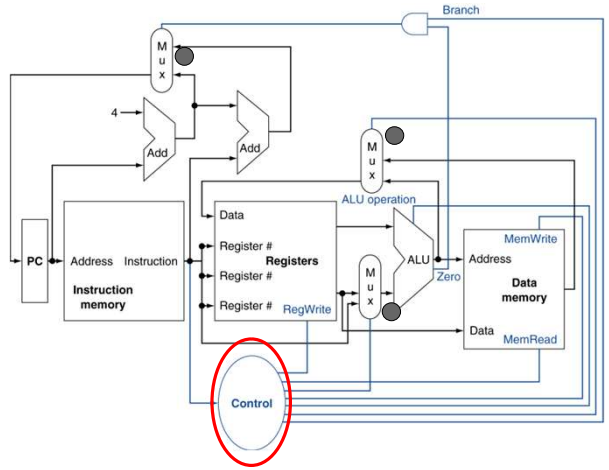
Multiplexers for stitching multiple paths



- Can't just join wires together
  - Use multiplexers

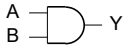
Control

IF → ID/RR → EX → Mem → WB

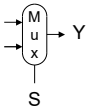


Combinational Elements

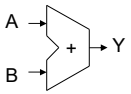
- **AND-gate**
  - **Y = A & B**



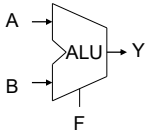
- Multiplexer



- Adder
  - Y = A + B

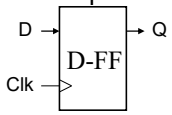


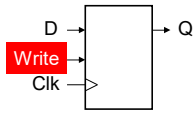
- Arithmetic/Logic Unit
  - Y = F(A, B)



Sequential Elements

- Register: stores data in a circuit (a chain of D-FFs)
  - Uses a clock signal to determine when to update the stored value
  - Edge-triggered: update when Clk changes from 0 to 1





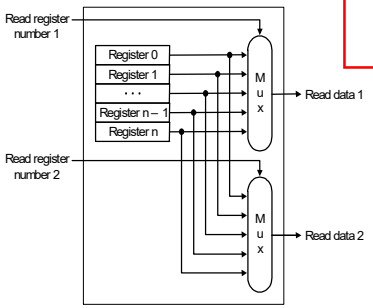
- Register with write control
  - Only updates on clock edge when write control input is 1
  - Used when stored value is required later

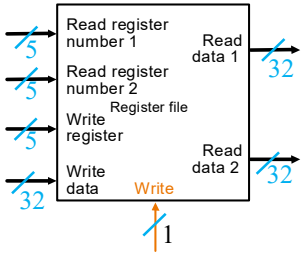
Register File: Read

- Built using D flip-flops

MIPS has a 32 × 32-bit register file
Use for frequently accessed data
Numbered 0 to 31

Assembler names
$t0, $t1, …, $t9 for temporary values
$s0, $s1, …, $s7 for saved variables

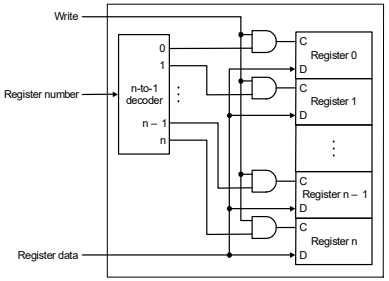




add rd, rs, rt

Register File: Write

- use the clock to determine when to write, provided *Write* is enabled



add rd, rs, rt